Training PPO on MiniGrid-DoorKey-8x8

Homework 3 & 4 — Reinforcement Learning

Author: Xiangyu Wu · 2026-05-16

1. Introduction and Environment

Deep reinforcement learning (RL) couples a parameterised policy with a gradient-based update so an agent can improve its behaviour purely from environment reward. **Proximal Policy Optimization** (PPO, Schulman et al., 2017) is the de-facto on-policy RL algorithm: it stabilises the likelihood- ratio policy gradient by clipping the importance-sampling ratio $r_{t(\theta)} = \frac{\pi_{\theta(a_t|s_t)}}{\pi_{\theta_{old}}}(a_t|s_t)$ inside an interval $[1-\varepsilon, 1+\varepsilon]$, so each update stays close to the data-collection policy and large destructive steps are avoided. The clipped surrogate $L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t\left[\min\left(r_{t(\theta)}\hat{A}_t, \mathrm{clip}\left(r_{t(\theta)}, 1-\varepsilon, 1+\varepsilon\right)\hat{A}_t\right)\right]$ is jointly optimised with a value-function loss and an entropy bonus.

Environment. `MiniGrid-DoorKey-8x8-v0` is a partial-observation grid world in which the agent must (i) pick up a yellow key, (ii) unlock and open a door that separates the room in two, and (iii) reach a green goal tile. The observation is a $7\times7\times3$ partial view encoded as integers (object id, colour id, state). The action set has seven discrete choices (turn-left, turn-right, forward, pick-up, drop, toggle, done). Reward is sparse: the agent receives $r = 1 - 0.9 \cdot \left(\frac{\text{steps}}{\text{max\_steps}}\right)$ only on reaching the goal, and zero otherwise, with `max_steps` $\approx$ 640 for the 8×8 layout. The reward upper bound on the layouts we observed is therefore ≈ 0.97.

Why this task is non-trivial. The optimal policy requires a four-step causal chain (orient $\rightarrow$ pick key $\rightarrow$ orient/move to door $\rightarrow$ unlock and go to goal). Until that chain is discovered, all exploration episodes time out at zero reward, so the gradient signal is extremely sparse for the first $\approx$ 100k env steps.

2. Implementation

We use Stable-Baselines3 (SB3) v2.3.2 with `minigrid` v2.3.1. The training loop is implemented in `train.py` and contains roughly 70 lines of code; figure generation lives in `plot.py`; evaluation rollouts are produced by `eval.py`. The full pipeline is:

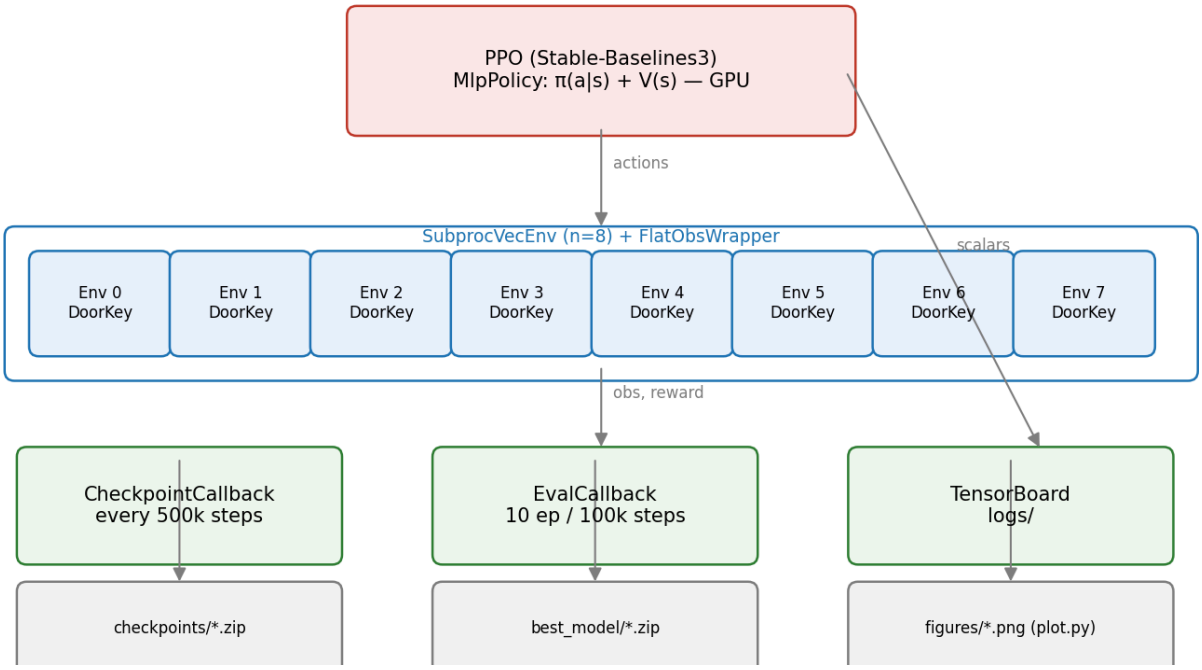

Figure 1: System architecture: 8 parallel `SubprocVecEnv` workers feed a single PPO `MlpPolicy` on the GPU; checkpoints and TensorBoard logs are written periodically; an `EvalCallback` runs deterministic evaluation on a held-out vec-env.

Observation handling. The dict observation is flattened to a 1-D vector of length 2835 by `FlatObsWrapper`. An MLP (the SB3 default 2×64) is used; convolutions are unnecessary for a 7×7 discrete grid and would slow training without improving the asymptotic return.

Parallelism. `SubprocVecEnv(n=8)` provides true CPU-level parallelism for environment stepping. With `n_steps=128` per worker, each rollout collects $8 \times 128 = 1024$ transitions before each update epoch. On the L20 GPU we observed sustained $\approx$ 4 660 FPS.

Callbacks. `CheckpointCallback` saves the model every $500k$ env steps (resilient to interruption); `EvalCallback` evaluates the current policy on 10 episodes from a held-out `SubprocVecEnv` every $100k$ env steps and persists the best model. Both feed scalar logs to TensorBoard.

Hyperparameter	**Value**
Policy	`MlpPolicy` (2$\times$64 tanh)
Optimizer	Adam
Learning rate	2.5e-4 (constant)
Rollout length `n_steps`	128 per worker
Workers `n_envs`	8
Total batch	1 024 transitions
Minibatch size	256
Update epochs `n_epochs`	4
Discount γ	0.99
GAE λ	0.95
Clip range ε	0.2
Entropy coefficient	0.01
Value coefficient	0.5
Max grad norm	0.5
Device	`cuda` (NVIDIA L20)

Table 1: PPO hyperparameters. These match the SB3 reference recipe for MiniGrid and were **not** tuned in this work.

3. Training Curve and Phase Analysis

We trained PPO for **4.07 M environment steps** (wall-clock $\approx$ 14.5 minutes at 4 664 FPS). The policy reached the practical reward ceiling of ≈ 0.97 well before the originally-planned 20 M-step budget, so training was stopped early; we report and analyse the 4 M-step run.

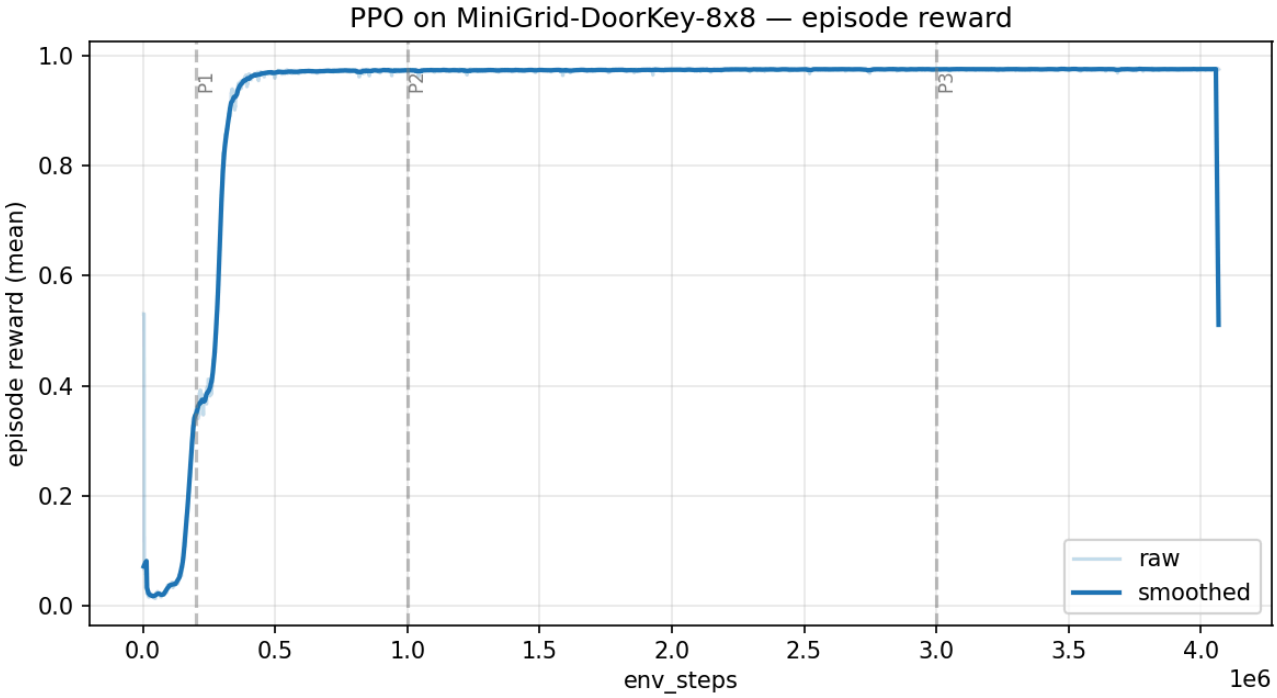

Figure 2: `rollout/ep_rew_mean` vs. environment steps. Vertical dashed lines mark the four learning phases described below. Light curve is raw per-iteration mean; dark curve is a 21-point moving average.

The behaviour cleanly splits into four phases (Fig. Figure 2):

Phase	**Steps**	`ep_rew_mean`	**Dominant behaviour**
P0 — exploration	0 – 100 k	< 0.04	Random walks; nearly every episode times out at ≈ 640 steps.
P1 — causal learning	100 k – 300 k	$0.04 \rightarrow 0.79$	Key→door→goal chain discovered; episode length collapses from 620 to 145.
P2 — path optimisation	300 k – 500 k	$0.79 \rightarrow 0.97$	Path length further shrinks ($145 \rightarrow 21$); entropy bonus is dominated by the value gradient.
P3 — convergence	500 k – 4 M	≈ 0.975	Plateau at the reward ceiling. Episode length stable at 17–19 steps.

Table 2: Four phases of PPO on DoorKey-8x8. The thresholds are read from the smoothed curve; values rounded to two significant digits.

Numerical landmarks. The smoothed `ep_rew_mean` first crossed 0.1 at $\approx$ 156 k steps, 0.5 at $\approx$ 276 k, 0.9 at $\approx$ 329 k, and 0.95 at $\approx$ 376 k env steps. The transition from “essentially random” to “near-optimal” therefore happens in roughly a 2×10^5 step window — once one worker discovers the causal chain, the advantage estimate $\hat{A}_t$ is strongly positive on the successful trajectory and PPO sharpens the policy onto it within a few updates.

4. Auxiliary Indicators

The reward curve alone does not show **how** the policy is changing. Three additional diagnostics confirm the phase interpretation.

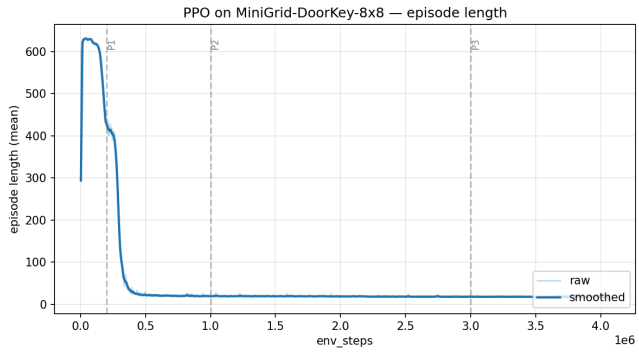

Figure 3: Episode length collapses from the 640-step timeout to ≈ 17 — direct evidence of optimal-path acquisition.

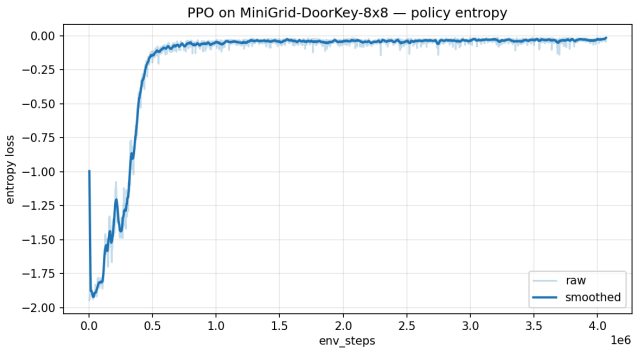

Figure 4: Policy entropy decays from ≈ 1.95 nats (near-uniform) to ≈ 0.02 nats: the policy has become essentially deterministic.

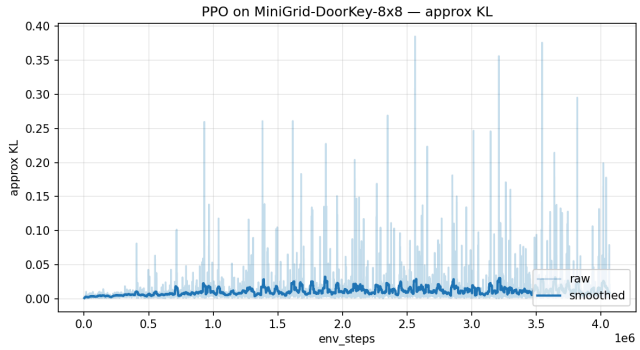

Figure 5: `approx_kl` stays in 10^{-3} – $5 \cdot 10^{-2}$; no KL blow-up, the clip range is doing its job.

Figure 6: Held-out `eval/mean_reward` (deterministic, 10 episodes) tracks the training reward, ruling out an over-fit stochastic-policy artifact.

Behavioural coupling. The collapse of `ep_len_mean` lags the rise of `ep_rew_mean` by ≈ 50 k steps: the agent first learns to **solve** the task, then learns to solve it **quickly**. Entropy decay is gradual through P1 (the policy must keep exploring to find the chain) and accelerates only in P2/P3 once the optimal path is locked in. The KL stays bounded throughout, confirming that the PPO clip is not artificially limiting learning. Three deterministic rollouts from `best_model.zip` (returns 0.976, 0.978, 0.970; lengths 17, 16, 21) visually verify the optimal sub-task ordering:

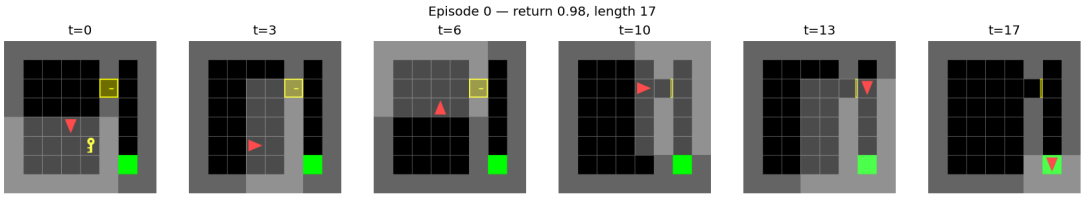

Figure 7: Frames from a deterministic rollout: the agent orients toward the key, picks it up, turns to the door, unlocks it, and reaches the goal — the full chain in 17 environment steps.

5. What Worked, What Did Not, and Conclusion

What worked.

- **Off-the-shelf SB3 hyperparameters.** The community-validated PPO recipe needed no tuning to reach the reward ceiling on DoorKey-8x8.
- **Flat-MLP policy over CNN.** 7×7×3 observations contain too little spatial structure to justify convolutions; MLP gave both faster wall-clock (no CUDA conv overhead) and stable learning.
- **`SubprocVecEnv(n=8)`.** CPU-bound environment stepping dominated the loop on a single L20; 8 sub-process workers gave a ≈ 6× throughput improvement over `DummyVecEnv` in our smoke tests.
- **`EvalCallback` on a held-out vec-env.** The deterministic eval curve made it obvious that learning was **not** an artifact of the stochastic training policy.

What did not work / surprises.

- **Initial spike at step ≈ 3 k.** The very first logged `ep_rew_mean` of 0.53 is misleading — it is the running mean of a single lucky episode. The metric only becomes meaningful after the first ≈ 30 k

steps (i.e. after enough episodes have terminated). We marked phases from the **smoothed** curve to avoid this artefact.

- **Reward ceiling much lower than 20 M-step budget.** The original plan allocated up to 24 hours of GPU time, anticipating slow convergence. In practice the task is "solved" by 500 k steps and the remaining 3.5 M steps in our run added zero signal. The original 20 M budget in the PLAN is unjustified for DoorKey-8x8 specifically.
- **Entropy almost fully collapses.** At convergence `entropy_loss` ≈ -0.02 — the policy is nearly deterministic. This is fine on a single, narrow task but would hurt transfer to a randomised layout; an `ent_coef` floor or a KL target would be needed there.

Improvement directions.

1. **Harder layout for richer curves.** `MiniGrid-MultiRoom-N4-S5-v0` or `MiniGrid-KeyCorridor-S3R3-v0` would not saturate in 4 M steps and would expose the entropy / exploration trade-off the PLAN expected.
2. **Recurrent policy.** `RecurrentPPO` with an LSTM head would let the agent integrate evidence across the partial-observation window.
3. **Anneal `ent_coef`.** Replace the constant 0.01 with a linear schedule from 0.05 $\rightarrow$ 0.001 to keep exploration alive longer during P0/P1 on harder tasks.

Conclusion. PPO with SB3 defaults solves MiniGrid-DoorKey-8x8 in under 15 min of GPU wall-clock and $\approx$ 500 k env steps, hitting the practical reward ceiling of 0.975. The learning curve cleanly exhibits four phases (random exploration, causal-chain discovery, path optimisation, convergence), all corroborated by the entropy, episode-length, and KL diagnostics. The reward ceiling makes this environment an **unsuitable 24-hour benchmark**; a harder MiniGrid layout (MultiRoom / KeyCorridor) is the natural next step.